

Contents

SYSTEM DESIGN CASE STUDY	2
Introduction & Executive Context	3
Executive Summary	3
The Problem Statement	3
Target State Architecture & Structural Design	5
Comprehensive Solution Summary	8
End-End Customer Journey	11
Transaction Data-Flows	14
Security Architecture	17
Disaster Recovery (DR) Plan	19
Architecture Tradeoffs	21
Architecture Decision Records	23
AWS Well-Architected Framework Review	26
Architecture Risks	28
Architectural Recommendations	31
Boundary Conditions for Reference Architecture	33
Appendix A: Non-Functional Requirements (NFR) Matrix	36
Appendix B: Cost Optimization & Attribution Matrix	38
Appendix C: Enterprise Security Risk Register	40
Appendix D: Enterprise RACI Matrix	43
About the Author	44

SYSTEM DESIGN CASE STUDY

Legacy MainFrame Hybrid Architecture on AWS

Prepared by:

Amit Kulkarni

Introduction & Executive Context

Modern financial institutions face a critical challenge: unlocking the value of core banking data locked inside legacy mainframes without disrupting daily business operations. This hybrid architecture pattern delivers a low-risk strategy to bridge the gap between traditional banking infrastructure and modern cloud applications.

By creating a secure, high-performance bridge between the *AWS Cloud* and an on-premises *IBM z/OS Mainframe*, organizations can deploy fast-moving fintech applications while keeping their core transactional systems of record intact.

Executive Summary

This architecture pattern defines a secure, high-performance hybrid cloud blueprint that bridges modern cloud-native applications with legacy core systems of record. By integrating the *AWS Cloud* platform with an on-premises *IBM z/OS Mainframe*, financial institutions and enterprise organizations can accelerate digital transformation without the high risk, cost, and complexity of a complete mainframe "rip-and-replace" migration. The solution utilizes *AWS ECS* and *Red Hat OpenShift* to deploy agile, customer-facing fintech services at the cloud edge, while leveraging the IBM Z and Cloud Modernization Stack (*IBM z/OS Connect*) to expose monolithic mainframe assets as secure, reusable RESTful APIs.

The Problem Statement

1. The Integration Barrier

- **Proprietary Data Architectures:** Legacy platforms like *IBM CICS* and *IMS* rely on rigid, binary data formats that modern web applications cannot read natively.
- **Monolithic Codebases:** Core business logic is deeply buried in decades-old *COBOL* or *PL/I* programs, making simple software updates risky and slow.
- **Protocol Incompatibility:** Modern cloud applications communicate via lightweight *JSON REST* APIs, whereas mainframes require specialized, low-level network protocols.

- **Developer Skill Gaps:** Cloud-native engineers lack experience with mainframe environments, while experienced assembly and mainframe engineers are rapidly retiring.

2. The "Rip-and-Replace" Migration Risk

- **Unacceptable Operational Downtime:** Dissecting and moving core transaction engines risks catastrophic system outages that can disrupt global business operations.
- **High Financial Investment:** Fully migrating a legacy mainframe system frequently costs tens of millions of dollars and takes years to complete.
- **Unpredictable Failure Points:** Monolithic systems have undocumented dependencies, meaning moving a single feature can break unrelated, critical business tasks.
- **Data Loss Risks:** Extracting, transforming, and loading complex transactional databases like *VSAM* and *IMS DB* into the cloud carries high risks of data corruption.

3. Prohibitive MIPS Scaling Costs

- **Expensive Processing Units:** Mainframe operational costs are tied directly to Millions of Instructions Per Second (*MIPS*), making raw compute power highly expensive.
- **Mobile Traffic Spikes:** Modern mobile banking and fintech applications generate millions of rapid, non-transactional read requests like balance lookups.
- **Inconsistent Infrastructure Costs:** Direct, uncached cloud queries pass directly to the mainframe, creating unpredictable and costly operational spikes during peak hours.
- **Wasteful Compute Allocation:** High-performance mainframe processors are wasted on simple data retrieval tasks instead of focusing on high-value transaction processing.

4. Siloed Data and Slow Innovation

- **Trapped Core Data Assets:** Valuable customer data is locked inside on-premises databases, blocking modern analytics tools and AI processing models.

- **Delayed Time-to-Market:** Launching a new digital product requires complex, multi-month changes across both cloud networks and mainframe systems.
- **Ecosystem Isolation:** Legacy systems cannot easily connect with external partner networks or modern third-party open-banking APIs.
- **Fragmented Development Pipelines:** Cloud developers use fast, automated DevOps tools while mainframe deployments rely on slow, manual release cycles.

Target State Architecture & Structural Design

System Architecture Overview

The target state architecture establishes a highly secure, resilient, multi-tier hybrid-cloud topology engineered to bridge modern *AWS cloud-native* microservices with an on-premises *IBM Z legacy mainframe* core. The system decouples internet-facing ingestion and edge routing layers from internal application runtimes to support heavy transactional processing, secure container orchestration, and high-performance mainframe data integration safely. All internal workflows run beneath a centralized Management & Security wrapper to ensure strict data governance, zero-trust perimeter isolation, and financial-grade enterprise compliance across independent availability zones.

Core Functional Processing Tiers

1. Multi-Source Edge Data Ingestion Tier

The data ingestion boundary interfaces with three segregated external source clusters to aggregate fintech, consumer, and partner transaction metrics:

- **Customers & App Channels:** Mobile, web, and consumer applications interact with the infrastructure by sending edge API requests down through automated application workflows.
- **Fintech Partners:** Third-party financial service providers and external open-banking platforms stream specialized data packages directly into the ingestion layers.
- **External Core Infrastructure:** Hybrid business partners and legacy customer networks route secure business transactions into the cloud ecosystem over dedicated infrastructure lines.

2. Managed Landing & Extraction Ingress Pipeline (Ingest Gateways)

Data crossing the enterprise cloud network perimeter passes into an optimized Ingest Gateway network layer featuring four specific extraction paths:

- **Global DNS Routing:** *AWS Route 53* manages external domain name resolutions and applies latency-based policies to route incoming network packets efficiently.
- **Edge Edge-Caching Network:** *AWS CloudFront* terminates TLS sessions close to users and serves static digital assets through globally distributed edge locations to minimize network latency.
- **Identity Management Gateway:** *AWS Cognito* handles identity federation, user pool management, and secure *JWT token* generation to gate enterprise application boundaries.
- **Traffic Distribution Engine:** A high-throughput *AWS Network Load Balancer* (NLB) maps incoming Layer 4 TCP/UDP traffic and distributes payloads evenly across multiple availability zones.

3. Containerized Application, Microservices, & Modernization Tier

The runtime processing ecosystem leverages a *multi-AZ* container layout to isolate stateless microservices from specialized legacy integration engines:

- **High-Availability Availability Zones:** The infrastructure duplicates all compute blocks across *Availability Zone A* and *Availability Zone B* to eliminate single points of failure and survive localized outages.
- **Containerized Microservices Cluster:** *AWS ECS* manages and autoscales independent, stateless Fintech Services packaged inside *Docker containers* to process core business logic.
- **Serverless Compute Nodes:** *AWS Lambda* executes stateless, event-driven function routines asynchronously to handle specialized computing workloads without provisioning fixed servers.
- **Mainframe Integration Stack:** *RedHat OpenShift* hosts the *IBM Z and Cloud Modernization Stack*, acting as a translation bridge that converts legacy mainframe structures into modern, cloud-native REST/JSON API matrices.

4. Relational Data, Caching, & On-Premises Core Output Tier

Modernized application data blocks are funneled through managed cloud databases or routed directly down into high-performance mainframe subsystems:

- **In-Memory Caching Stratum:** *AWS ElastiCache* manages high-speed session storage and caches frequently accessed records to offload heavy read footprints from underlying databases.
- **Managed Cloud Storage & DB:** *AWS S3* captures immutable application data logs and backups, while *AWS RDS* processes managed relational database schemas inside isolated *private subnets*.
- **Dedicated Enterprise Pipelines:** *AWS Direct Connect* establishes a dedicated physical fiber link to transport core data streams to the mainframe securely, while an *AWS VPN Connection* serves as an encrypted, air-gapped backup path.
- **Legacy System of Record:** An On-Premise Data Centre runs an *IBM z/OS Mainframe* cluster that anchors critical transactional engines (*CICS TS* and *IMS TM*) alongside core transactional databases (*IBM Db2*, *IMS DB*, and *VSAM* files).

5. Centralized Management, Security, & Governance Compliance Wrap

Continuous zero-trust enforcement, data protection, and proactive security operations run globally across the entire active framework using seven core components:

- **AWS Identity & Access Management (IAM):** Mandates granular, role-based access controls (RBAC) to ensure strict least-privilege service-to-service execution boundaries between *AWS ECS*, *AWS Lambda*, and *AWS RDS*.
- **AWS Key Management Service (KMS):** Enforces global data isolation by forcing *AES-256* standard encryption-at-rest with automated key rotation across all *AWS S3* tiers, *AWS ElastiCache* nodes, and *AWS RDS* databases.
- **AWS WAF (Web Application Firewall):** Inspects inbound Layer 7 web traffic dynamically to block malicious payloads, *SQL injections*, and *Cross-Site Scripting (XSS)* vectors.
- **AWS Shield:** Provides always-on, managed Layer 3 and Layer 4 *DDoS protection* to shield the entry network perimeter from infrastructure-level volumetric attacks.

- **Amazon GuardDuty:** Runs continuous, intelligent machine learning analysis across network logs and *CloudTrail* transactions to flag infrastructure anomalies and detect edge threats.
- **AWS CloudTrail:** Registers an unalterable, cryptographically validated ledger of all cloud system API transactions to maintain complete regulatory audit tracking for financial compliance.
- **Amazon CloudWatch:** Aggregates centralized infrastructure logs, container health metrics, and automated exception alarms to provide proactive observability across the hybrid cloud mesh.

Comprehensive Solution Summary

Comprehensive Solution Summary

This enterprise-grade, production-ready solution delivers a highly resilient, hybrid-cloud application modernization architecture designed in strict compliance with the *AWS Reference Architecture for Mainframe Modernization*. Engineered to resolve legacy application fragmentation, eliminate on-premises operational silos, and secure public-facing endpoints, this solution acts as a unified platform for multi-source ingestion, secure container orchestration, real-time API transformation, and core system-of-record integration. By decoupling internet-facing ingress layers from high-performance backend transactional frameworks, the platform guarantees predictable scalability and consistent operational delivery while processing volatile workloads of enterprise financial transactions and cloud-native fintech services.

1. End-to-End Managed Data Ingestion & Extraction Ingress Framework

The platform's data entry boundary isolates and aggregates traffic arrays from distinct consumer, provider, and fintech institutional environments.

- **Edge Protection and Global CDN Performance:** Public-facing client requests and fintech application payloads route globally through *AWS Route 53* and *AWS CloudFront* to terminate TLS sessions at edge locations, drastically reducing network latency. Concurrently, *AWS WAF* and *AWS Shield* scrub incoming *Layer 7* traffic to block malicious web exploits, *SQL injections*, and volumetric *DDoS* attacks.
- **Identity Federation and Boundary Gating:** User authentication and third-party access clear through *AWS Cognito*, which manages secure identity user pools and issues short-lived *JWT validation* tokens. This enforces rigid

perimeter access controls and verifies tenant permissions before requests penetrate deeper into the internal network.

- **High-Throughput Traffic Distribution:** Authenticated network streams enter a multi-tier *Virtual Private Cloud* (VPC) where an *AWS Network Load Balancer* (NLB) maps incoming Layer 4 TCP/UDP connections. The NLB distributes compute payloads symmetrically across isolated private subnets to maintain high availability and protect backend compute resources from public exposure.

2. Multi-Tier Container Compute, API Translation, and Hybrid Core Ecosystem

The core computing ecosystem utilizes a *multi-AZ* container and translation topology to completely decouple modern cloud runtimes from legacy on-premises mainframe transactional frameworks.

- **Decoupled Microservices and Serverless Compute Foundation:** Core application business logic executes inside an isolated, highly available runtime environment organized across two distinct processing tiers:
 - **Containerized Service Zone:** *AWS ECS* hosts and autoscales independent, stateless fintech microservices packaged inside Docker containers to process high-velocity application logic across independent *availability zones* (AZ-A and AZ-B).
 - **Serverless Execution Zone:** *AWS Lambda* executes event-driven, asynchronous routines on demand, minimizing fixed server overhead for non-continuous operational workloads and background file tasks.
- **Modernization Stack and API Transformation:** *RedHat OpenShift* hosts the *IBM Z and Cloud Modernization Stack*, acting as a real-time integration bridge. This layer compiles and translates cloud-native *JSON/REST* API calls directly into legacy mainframe data streams and structures (such as *COBOL copybooks* or *EBCDIC matrices*), eliminating the need for cloud services to understand native mainframe protocols.
- **In-Memory Caching and Private Storage Boundaries:** *AWS ElastiCache* manages high-speed session states and intercepts read-heavy transaction volumes to shield backend databases from redundant queries. Persistent relational schemas are isolated within private database subnets managed by *AWS RDS*, while *AWS S3* captures immutable transaction logs and infrastructure backups.
- **Redundant Hybrid Connectivity and Legacy Execution Core:** Crucial transaction payloads travel across a dedicated physical, low-latency *AWS*

Direct Connect (DX) pipeline, which is backed by an air-gapped *AWS VPN* Connection via dynamic *BGP routing* for automated failover. These lines deliver requests straight into the on-premises *IBM z/OS Mainframe*, where core transaction monitors (*CICS TS* and *IMS TM*) securely execute updates against legacy database clusters (*IBM Db2*, *IMS DB*, and *VSAM* files).

3. Centralized Zero-Trust Governance, Security Wrapping, and Compliance Framework

This security architecture deploys an absolute zero-trust paradigm across all cloud boundaries and hybrid communication paths to enforce deep identity protection, real-time threat intelligence, and strict financial regulatory compliance.

- **Granular Least-Privilege Identity Controls:** *AWS Identity & Access Management (IAM)* mandates rigid, role-based access controls (*RBAC*) and explicit service-to-service execution policies, strictly isolating containerized microservices, *Lambda* execution contexts, and *AWS RDS* configurations to prevent lateral movement.
- **Global Cryptographic Data Isolation:** *AWS Key Management Service (KMS)* coordinates centralized envelope encryption across all active storage zones, enforcing *AES-256* standard encryption-at-rest with automated key rotation routines across *AWS S3* log buckets, *AWS ElastiCache* nodes, and relational database clusters.
- **Continuous Multi-Layer Threat Auditing & Metrics:** The hybrid mesh perimeter is continuously scrubbed and audited by automated threat-detection engines that aggregate real-time infrastructure telemetries:
- **Amazon GuardDuty:** Executes advanced machine learning behavioral analysis across network flow tables and DNS queries to flag anomalous activity and isolate active edge vulnerabilities.
- **AWS CloudTrail & Config:** *CloudTrail* registers an unalterable, cryptographically validated ledger of all system API transactions, while *AWS Config* tracks live resource configurations to guarantee strict adherence to financial auditing benchmarks.
- **Amazon CloudWatch:** Aggregates centralized container logs, API exception alarms, and cluster performance vitals to deliver end-to-end operational observability across the enterprise cloud fabric.

4. Hybrid Network Topography, Redundant Connectivity, and High-Availability Failover Fabric

The network infrastructure is engineered to sustain severe physical hardware disruptions and localized carrier outages while maintaining continuous, low-latency transaction routing between public cloud endpoints and on-premises core data centers.

- **Isolated Virtual Private Perimeter:** The structural landscape segregates public-facing edge layers from sensitive compute and database engines by confining all container clusters, API translation stacks, and relational schemas inside private subnets across multiple separate *Availability Zones* (AZ-A and AZ-B).
- **Deterministic Dual-Path Core Transit Routing:** Connectivity to the on-premises enterprise core leverages a dual-carrier hybrid configuration to bypass public internet exposure entirely:
- **Primary High-Bandwidth Dedicated Pipeline:** *AWS Direct Connect* (DX) establishes a physical, dedicated fiber-optic cross-connect to route critical synchronous core transaction workloads with single-digit millisecond latency directly to the data center cage.
- **Air-Gapped Encrypted Standby Tunnel:** An *AWS VPN* Connection acts as an active-standby *IPsec tunnel* that continuously monitors connection state vitals through dynamic *Border Gateway Protocol* (BGP) routing over alternative paths.
- **Automated Failure Remediation Fabric:** If a primary fiber trunk experiences a physical cut or a specific availability zone degrades, the *Network Load Balancer* (NLB) and *BGP routing* layers execute instantaneous, sub-second failover pathing, redirecting active transaction loads to surviving availability zones or the secondary *VPN gateway* without losing transactional state.

End-End Customer Journey

1: Edge Ingress, Perimeter Scrubbing, and Content Delivery

- **The User Action:** A banking customer logs into their mobile application and initiates a secure fund transfer request.
- **Amazon Route 53:** The mobile client dispatches an initial *DNS* lookup. *AWS Route 53* evaluates the incoming request using latency-based routing policies to pinpoint the absolute closest entry node.

- **Amazon CloudFront:** The client establishes a secure connection with a *AWS CloudFront* Edge location. *CloudFront* terminates the *TLS* session close to the user, accelerating the initial handshake and serving static cached assets locally to reduce overall network latency.
- **AWS WAF & Shield:** Before any packet traverses deep into the infrastructure, *AWS WAF* inspects inbound Layer 7 payload strings to detect and block malicious web exploits, *SQL injections*, and *Cross-site Scripting* (XSS) anomalies. Concurrently, *AWS Shield* provides always-on, managed Layer 3/4 protection to isolate the network from volumetric *DDoS* attacks.

Step 2: Federated Identity Validation and Perimeter Access Gating

- **Amazon Cognito Identity Provider:** The incoming request payload presents an *OAuth 2.0* security token. *AWS CloudFront* redirects this specific identity challenge to *AWS Cognito*.
- **JWT Cryptographic Verification:** *Cognito* evaluates the cryptographically signed *JSON Web Token* (JWT) against predefined User Pools. It verifies active user session signatures, cross-references explicit client permissions, and attaches a verified, short-lived security context to the request.
- **Traffic Gating:** Unauthenticated or malformed requests are instantly dropped at this boundary, ensuring that only verified, clean connection states are permitted to cross the enterprise cloud perimeter.

Step 3: High-Velocity Load Balancing and Private Subnet Ingress

- **VPC Perimeter Ingress:** The authorized packet passes through an internet gateway into the secure boundary of an isolated multi-tier *Virtual Private Cloud* (VPC).
- **AWS Network Load Balancer (NLB):** The network payload lands directly on an *AWS NLB*. Working strictly at Layer 4 (Transport Layer), the *NLB* processes millions of concurrent connections with microsecond latency.
- **Multi-AZ Distribution:** The *NLB* maps incoming *TCP* streams and distributes the application payloads symmetrically across isolated *private subnets* divided between two independent availability zones (*Availability Zone A* and *Availability Zone B*) to prevent single points of failure.

Step 4: Container Execution and In-Memory Session Inspection

- **Amazon ECS Fintech Services:** The *NLB* forwards the transaction packet inside a private *compute subnet* to an *AWS ECS* cluster running containerized, stateless fintech microservices.
- **Amazon ElastiCache:** Before making costly downstream calls, the *AWS ECS container* queries an in-memory *AWS ElastiCache* cluster. It instantly pulls the user's active session state, profile configurations, and recent ledger metadata in sub-milliseconds, avoiding redundant database lookups.
- **Amazon RDS Execution:** For secondary persistent cloud operations (such as auditing local app preferences or verifying local cloud balances), the *AWS ECS* service reads from an isolated *AWS RDS* relational database instance configured with synchronous *cross-AZ* replication.

Step 5: Asynchronous Decoupling and Event-Driven Tasking

- **AWS Lambda Integration:** To ensure the main web application thread stays entirely non-blocking, the *AWS ECS container* offloads non-critical, ancillary workflows (such as firing off *SMS* push alerts, triggering transaction receipt generations, or scheduling fraud check loops) to event-driven *AWS Lambda* functions.
- **Serverless Compute Runs:** *Lambda* provisions serverless compute nodes on demand to execute these asynchronous background processing functions independently, shutting down automatically upon execution completion to eliminate idle compute costs.

Step 6: Protocol Translation and Legacy Payload Compilation

- **RedHat OpenShift Modernization Stack:** To communicate with a legacy core banking ledger, the modern *JSON* payload must be structurally transformed. The *AWS ECS* service passes the request via a high-performance gRPC call to the *IBM Z* and Cloud Modernization Stack running on *RedHat OpenShift* inside the *private subnet*.
- **JSON-to-EBCDIC Mapping:** The modernization stack parses the lightweight *JSON* fields and uses low-latency compilation middleware to re-encode the data structures. It transforms modern *ASCII* strings into legacy *EBCDIC* text matrices and compiles the variables directly into binary structures matching a native *COBOL Copybook layout*.

Step 7: Deterministic Hybrid Transit and Core Mainframe Commit

- **AWS Direct Connect (DX):** The translated binary transaction payload travels out of the *AWS private cloud perimeter* over a dedicated, physical fiber-optic *AWS Direct Connect* pipeline routing directly into the on-premises enterprise data center cage with single-digit millisecond latency.
- **Active-Standby VPN Failover:** If a physical fiber line cut occurs, dynamic *Border Gateway Protocol* (BGP) routing triggers an automated, sub-second failover, instantly rerouting active transaction payloads over an air-gapped, encrypted *AWS VPN* Connection tunnel.
- **IBM z/OS Mainframe Processing:** The payload enters the inner core of the on-premises mainframe. *CICS TS* (Customer Information Control System) captures the execution thread, locks the target account balance, updates the system of record inside an *IBM Db2* or *IMS DB* cluster, commits changes to *VSAM* logging files, and returns a verified success block back up the pipeline.

Step 8: Comprehensive Compliance Ledgering, Observability, and Audit Caching

- **Response Return:** The successful execution matrix travels back up through the hybrid network. The *OpenShift* stack compiles it back into a modern JSON 200 OK structure, and the *AWS ECS* container fires the response back to the client's mobile screen.
- **AWS CloudTrail & Config:** As the transaction completes, an unalterable, cryptographically validated record of every single API transaction across all AWS services is committed to *AWS CloudTrail* for strict financial regulatory compliance auditing. Simultaneously, *AWS Config* continuously validates that no security group rules or encryption parameters were altered during processing.
- **Amazon CloudWatch:** Complete end-to-end telemetry (including container CPU trends, API exception logs, and network transit latency matrices) flows into *AWS CloudWatch*, giving operations teams real-time dashboards to observe system health proactively.

Transaction Data-Flows

This section maps the structural journey into discrete, execution-based data flows. It separates real-time, synchronous transaction processing from non-blocking background logic and persistent analytical data mirroring..

Flow 1: Synchronous Core Transaction Flow (Real-Time Execution Loop)

This flow processes immediate, mission-critical financial requests where the customer requires a real-time validation response.

Ingress Protocol: The customer's mobile application sends an encrypted HTTPS POST request containing a payload to initiate a fund transfer.

Edge Validation: *AWS Route 53* resolves the domain, passing the packet through *AWS CloudFront*. *AWS WAF* inspects the HTTP headers for injection strings, and *AWS Cognito* verifies the signed *JWT token*.

Internal Routing: The request passes through the *Network Load Balancer* (NLB) at Layer 4, which handles the TCP stream and shifts it to an active Transfer-Service task inside the *AWS ECS* private cluster.

Cache & Cloud State Check: The *AWS ECS* service issues a sub-millisecond read call to *AWS ElastiCache* to fetch user profile metadata and queries *AWS RDS* to verify application-specific parameters.

API Compilation: The service passes the verified *JSON* object to the *RedHat OpenShift* modernization cluster, which runs translation middleware to re-encode the data array from *ASCII* to *EBCDIC* and maps the layout directly into a binary *COBOL Copybook* configuration.

Mainframe Commit: The compiled payload traverses the *AWS Direct Connect* fiber link, reaching the on-premises *IBM Z Mainframe*. *CICS TS* acquires a ledger lock, alters the balance inside *IBM Db2*, records structural updates within *VSAM* indexing logs, and returns a binary success response code back up the line.

Client Delivery: The *OpenShift* stack converts the success flag back into a clean 200 OK *JSON* response, pushing it through the *NLB* and *CloudFront* edge layers to update the customer's mobile application screen.

Flow 2: Asynchronous Event-Driven Flow (Non-Blocking Background Tasks)

This flow isolates secondary operational actions from the primary transaction path, preventing downstream performance spikes from slowing down user response times.

Event Trigger: As soon as the synchronous transaction receives a validation confirmation from the *mainframe*, the *AWS ECS* container dispatches an internal event notification.

Compute Hand-off: Instead of waiting for secondary processing steps to complete, the container immediately drops the event package into an execution pipeline, freeing up its threads to take on the next inbound user request.

Serverless Invocation: *AWS Lambda* spins up isolated serverless container instances instantly to process the decoupled message payload.

Downstream Execution: The *Lambda* function executes parallel execution steps, It makes an API call to external notification gateways to push real-time *SMS* alerts and mobile device notifications directly to the customer. It generates a cryptographically signed digital receipt PDF, writing the persistent artifact out to an encrypted *AWS S3* object bucket.

Flow 3: Change Data Capture & Mirroring Flow (Data Lakehouse Ingestion)

This flow captures updates made inside the on-premises environment and mirrors them directly up to cloud storage systems to support reporting, auditing, and analytics workloads.

Transaction Event: A core balance update or account modification settles successfully inside the on-premises mainframe databases (*IBM Db2* or *IMS DB*).

Log Trapping: Middleware agents tracking the mainframe database transaction logs capture these low-level modifications as a real-time stream of data mutations.

Cloud Mirror Transit: The captured delta matrices are compressed, encrypted, and piped continuously across the *AWS Direct Connect* link to bypass public internet exposure entirely.

Landing Ingestion: The incoming data stream enters the *AWS Virtual Private Cloud*, where *AWS Database Migration Service* (*AWS DMS*) captures the continuous *Change-Data-Capture* (*CDC*) strings and feeds them directly into chronological raw folders within an *AWS S3 data lake* repository.

Analytics Ready Storage: Automated ingestion workflows pick up these raw delta records, clean them, convert them into query-optimized columnar formats (such as *Apache Parquet*), and store them within an *Analytics Storage Zone* on *S3*—making the updated financial data completely accessible for analytical querying engines and business intelligence dashboards.

Flow 4: Analytical Processing, Machine Learning, & Consumer Reporting Flow

This flow governs how the mirrored, query-optimized datasets inside the cloud ecosystem are consumed by data scientists, business analysts, and regulatory bodies without placing any operational load on the primary transaction engines.

1. **Schema Discovery:** As new *Change-Data-Capture* (*CDC*) blocks settle inside the *Analytics Storage Zone* of *Amazon S3*, scheduled *AWS Glue* Crawlers automatically scan the storage boundaries to detect structural

modifications and map updates directly into the centralized *AWS Glue Data Catalog*.

2. **Access Governance Enforcements:** Before any analytical compute engine can interface with the newly cataloged datasets, *AWS Lake Formation* evaluates the request against central data governance policies, enforcing granular, row- and cell-level access constraints to isolate sensitive customer identifiers.
3. **Ad-Hoc & Data Warehouse Analytics:** Business analysts execute serverless, schema-on-read queries directly across the *AWS S3* storage layers via *AWS Athena* for immediate ad-hoc research, while high-scale enterprise intelligence queries are processed within optimized multi-dimensional *AWS Redshift* data warehouses.
4. **Interactive Reporting Delivery:** *Amazon QuickSight* extracts records directly from the *Athena* and *Redshift* queries to populate interactive operational reporting dashboards, serving end-user business profiles and satisfying mandatory compliance visibility for state regulators.
5. **Predictive Machine Learning Modeling:** Simultaneously, enterprise Data Scientists utilize compute-optimized *AWS EC2* instances paired with *AWS SageMaker* to pull historical transactional vectors from high-performance shared *AWS EFS* file systems, training predictive models to detect complex cross-account fraud patterns automatically.

Security Architecture

This section outlines the defense-in-depth framework designed to protect customer financial data as it moves between the cloud and the mainframe.

1. Data Encryption & Key Management

- **Dual-Layer Transit Encryption:** All public traffic reaching the edge is encrypted using *TLS 1.3*, while private backend traffic traversing the *AWS Direct Connect* pipeline into the *Customer Gateway (CGW)* is wrapped in an additional layer of *IPsec VPN* encryption.
- **Cloud Envelope Encryption:** Data residing in *AWS RDS*, *AWS ElastiCache*, and *AWS S3* is encrypted at rest using *AWS Key Management Service (KMS)*, utilizing customer-managed keys (CMKs) that are automatically rotated annually.
- **Mainframe Hardware Cryptography:** Cryptographic operations on the *IBM z/OS* platform are offloaded to physical *IBM Crypto Express* adapters,

executing high-speed, hardware-enforced *AES-256* decryption of incoming cloud data packets.

- **Separation of Key Domains:** Cryptographic keys used in the *AWS Cloud* environment are completely isolated from the master keys governing the on-premises *IBM Db2* and *VSAM* storage layers, preventing a single platform compromise from exposing both environments.

2. Fine-Grained Access Control & Governance

- **Context-Aware API Gateway Policies:** *IBM z/OS Connect* running on *Red Hat OpenShift* evaluates requests based on dynamic variables—such as source IP, timestamp, and payload size—before mapping the *cloud JSON* request to mainframe business logic.
- **System-Level Security Mapping:** Incoming *REST API* requests are systematically mapped to native mainframe security profiles using *IBM RACF* (Resource Access Control Facility), ensuring every cloud transaction runs under a strictly governed mainframe user ID.
- **Transactional Micro-Segmentation:** The architecture divides the cloud *VPC* into isolated network zones. Databases like *AWS RDS* sit in isolated private *subnets*, explicitly rejecting all traffic except direct database queries from the *AWS ECS Fintech Services* tier.
- **Immutable Audit Trail Log Consolidation:** System activity logs from *AWS CloudTrail* and *IBM z/OS System Management Facilities (SMF)* are streamed continuously to a central log server to prevent tamper-prone administrative overrides.

3. Identity & Least Privilege Access Management

- **Token-Based Authentication:** *AWS Cognito* serves as the public-facing identity management platform. It authenticates users at the cloud edge and issues short-lived *JSON Web Tokens (JWTs)* that expire automatically after 15 minutes.
- **Role-Based AWS IAM Execution:** Applications executing within *AWS ECS* are assigned specific *IAM Task Roles*. These containers are denied root server permissions, allowing them only to fetch required secrets or connect to *AWS ElastiCache*.
- **Mainframe Transaction Isolation:** Subsystems like *CICS TS* and *IMS TM* enforce least privilege by restricting the cloud's service accounts. Service

accounts can only invoke predefined programs and are entirely blocked from accessing direct command lines.

- **Credential Elimination via Secret Management:** No static database passwords or mainframe main access keys are hardcoded into container code. The Fintech Services fetch rotational database credentials dynamically using encrypted API endpoints.

4. Boundary Protection & Network Security (Additional Section)

- **Automated Edge Threat Mitigation:** *AWS WAF* blocks SQL injection and cross-site scripting attempts at the entrance of the architecture, while *AWS Shield* provides automated mitigation against high-volume DDoS attacks.
- **Private Cloud Service Endpoints:** Cloud components communicate internally using private *VPC Endpoints*. This configuration ensures traffic bound for services like *AWS S3* stays entirely within the internal network, completely bypassing the public internet.
- **Strict Ingress Inspection:** The *Network Load Balancer* (NLB) only opens ports 443 and 80 to public edge components, immediately dropping unauthorized network discovery or port scanning traffic.
- **Unidirectional Connection Paths:** The hybrid connection is configured so that only the cloud-native applications can initiate an API handshake into the on-premises mainframe environment. The mainframe is blocked from initiating unexpected outbound connections back into the cloud network.

Disaster Recovery (DR) Plan

This plan ensures business continuity, system availability, and zero data corruption if an infrastructure failure or natural disaster takes down a cloud availability zone or the on-premises data centre.

1. Recovery Objectives & Design Standards

- **Strict RTO and RPO Targets:** The architecture targets a Recovery Time Objective (RTO) of under 4 hours for full system restoration and a Recovery Point Objective (RPO) of near-zero for financial accounting layers.
- **Geographic Cloud Redundancy:** AWS infrastructure runs across multiple isolated *Availability Zones* (AZs) by default, backed by a Pilot Light deployment in a secondary geographical *AWS region* to survive localized cloud blackouts.

- **On-Premise Parallel Active-Passive Site Alignment:** The legacy on-premises environment uses a primary data centre matched to a geographically distant secondary DR site, enabling immediate workload shift if the primary building loses power.

2. Cloud-Tier Failover Automation (AWS Engine)

- **Instantaneous DNS Failover Routing:** *AWS Route 53* monitors backend health check endpoints continuously. If an availability zone drops, it instantly re-routes 100% of user traffic to healthy infrastructure pathways without human intervention.
- **Cross-AZ Database Mirroring:** *AWS RDS* executes synchronous replication across *Availability Zones A* and *B*, performing automatic failover to the standby database copy within 60 seconds if the primary database engine fails.
- **Auto-Scaling Container Restorations:** *AWS ECS* tasks scale automatically across remaining zones. If an entire availability zone goes offline, AWS handles the deployment of containerized Fintech Services into remaining safe subnets.
- **Distributed Memory Cache Persistence:** *AWS ElastiCache* clusters are deployed with Multi-AZ replication enabled, allowing a read-replica node to be promoted to primary instantly if the main caching node crashes.

3. Hybrid Connectivity & Route Redundancy

- **Dual-Path Network Failover:** The primary data pipeline relies on an *AWS Direct Connect* dedicated physical line. If a physical line is severed, the system shifts traffic instantly to a backup encrypted *AWS Site-to-Site VPN* connection.
- **BGP Autonomous Re-Routing:** Enterprise routers employ *Border Gateway Protocol (BGP)* with active-passive path attributes, forcing networking hardware to re-route cloud payloads to the secondary data center path within seconds of a link drop.
- **Redundant API Transformation Hosting:** The IBM Z and Cloud Modernization Stack running on Red Hat OpenShift is deployed as an active cluster across all availability zones, ensuring protocol translation services stay online.

4. Mainframe-Tier Data Synchronization & Recovery

- **Real-Time Storage Mirroring:** Core on-premises mainframe storage updates (including *IBM Db2* and *VSAM*) are duplicated to the secondary DR site continuously using high-speed hardware replication links.
- **Mainframe Subsystem Fast-Boot Scripts:** If a complete primary data centre failure occurs, automation scripts initialize *CICS TS* and *IMS TM* transaction groups at the secondary site, restoring core banking logic in minutes.
- **Transactional Ledger Consistency Checking:** During a failover event, database recovery utilities scrub logs to isolate and complete partially processed cloud-to-mainframe API writes, ensuring no financial accounting errors slip through.

Architecture Tradeoffs

1. Agility vs. Multi-Platform Latency

- **The Benefit:** Offloading digital applications to *AWS ECS* and *Red Hat OpenShift* enables developers to build features rapidly using modern CI/CD pipelines, accelerating time-to-market.
- **The Trade-off:** Introducing multiple network hops—from *AWS CloudFront*, through the *NLB*, into *OpenShift*, across *AWS Direct Connect*, and finally into *CICS/IMS*—adds unavoidable network latency.
- **The Mitigation:** Highly efficient JSON-to-binary serialization via *IBM z/OS Connect* and aggressive caching of non-transactional lookups in *AWS ElastiCache* keep the total end-to-end response time well within standard enterprise SLAs.

2. MIPS Cost Reduction vs. Cloud Infrastructure Spend

- **The Benefit:** Intercepting high-volume, repetitive read requests (like balance inquiries) at the cloud edge using *AWS ElastiCache* and local *Amazon RDS* databases prevents expensive *MIPS* scaling costs on the mainframe.
- **The Trade-off:** Financial savings on the mainframe are partially offset by the ongoing operational costs of running a highly available, *Multi-AZ* *AWS* infrastructure. This includes dedicated *AWS Direct Connect* lines, enterprise *Red Hat OpenShift* licensing, and data transfer fees.

- **The Mitigation:** The organization shifts from an unpredictable, highly volatile *OpEx* model (mainframe *MIPS* spikes during peak hours) to a predictable, easily scalable cloud infrastructure spend profile.

3. System Decoupling vs. Operational Complexity

- **The Benefit:** Cloud microservices are decoupled from core mainframe monolithic code. Engineers can modify frontend services without touching or risking the stability of legacy core ledgers.
- **The Trade-off:** Troubleshooting system failures becomes significantly more complex. When an API transaction fails, engineers must trace the request across completely different logging ecosystems, spanning *AWS CloudWatch/CloudTrail* and on-premises *IBM System Management Facilities (SMF)*.
- **The Mitigation:** Implementing an end-to-end distributed tracing tool (such as *OpenTelemetry* integrated with mainframe monitoring agents) provides a unified view, tracking a single transaction ID from the mobile app down to the specific *IBM Db2* or *IBM VSAM* write.

4. Direct Database Access vs. Security & Governance Overhead

- **The Benefit:** Exposing mainframe subsystems strictly through controlled REST APIs governed by *IBM RACF* and *z/OS Connect* ensures that core banking data is never exposed directly to the public internet.
- **The Trade-off:** This strict layer of security introduces heavy governance and configuration overhead. Every single new data field or legacy function exposed to the cloud requires explicit mapping, protocol translation rules, IAM role configurations, and network firewall policy updates.
- **The Mitigation:** While it slows down initial API design phases, this strict control model guarantees that the enterprise maintains an uncompromised, zero-trust security posture and complies fully with stringent financial regulations (like *PCI-DSS*).

5. Single-Region Data Sovereignty vs. Hybrid Distributed Consistency (Dual-Write Risk)

- **The Benefit:** Maintaining the primary system of record on the on-premises *IBM z/OS Mainframe* guarantees strict ACID (Atomicity, Consistency, Isolation, Durability) compliance for financial ledgers, ensuring data integrity is never compromised at the core.

- **The Trade-off:** Storing synchronized or cached subsets of user data in cloud-native layers like *AWS RDS* or *AWS ElastiCache* creates a distributed data dilemma. If a network blip occurs over the *AWS Direct Connect* line during a complex multi-step transaction, the cloud database and the on-premises mainframe can fall out of sync, creating split-brain data anomalies.
- **The Mitigation:** The architecture enforces a strict "Mainframe-as-Master" operational rule. The cloud layer handles read-only queries from its local cache but never executes local database writes for core transactional states. Any state-changing transaction is written directly to the mainframe first, which then triggers an asynchronous event or cache invalidation to update the cloud-tier databases safely.

Architecture Decision Records

1. API-Driven Abstraction Layer via IBM z/OS Connect

- **Decision:** Expose mainframe transaction systems (*CICS TS* and *IMS TM*) strictly through *IBM z/OS Connect* running on *Red Hat OpenShift* within AWS, rather than allowing direct TCP/IP sockets or custom low-level connectivity from cloud containers.
- **Rationale:** This decouples cloud developers from proprietary mainframe data layouts. It automatically translates lightweight cloud JSON payloads into COBOL copybooks, transforming legacy mainframe components into standard, reusable RESTful APIs without modifying the core legacy code.

2. Local Read-Caching via Amazon ElastiCache

- **Decision:** Implement an *AWS ElastiCache* (Redis) tier within the cloud VPC to intercept and fulfill high-frequency, non-transactional read queries (e.g., account balance lookups).
- **Rationale:** Directly querying the mainframe for millions of daily mobile read actions drives up Millions of Instructions Per Second (*MIPS*) usage, causing massive financial cost spikes. Local cloud caching drastically lowers operational overhead and delivers single-digit millisecond response times to users.

3. Mainframe-as-Master for Data Consistency

- **Decision:** Establish the on-premises *IBM z/OS* Mainframe databases (*IBM Db2, IMS DB, VSAM*) as the ultimate single source of truth (SSOT). The cloud layer (*AWS RDS*) can only store non-critical user state data or read-only replicas.
- **Rationale:** Financial and core enterprise data requires strict ACID compliance. Forposing a dual-write model across both environments introduces high data corruption risks during network drops. Any state-altering write must commit to the mainframe ledger first.

4. Dedicated Hybrid Networking via AWS Direct Connect with VPN Backup

- **Decision:** Utilize physical *AWS Direct Connect* as the primary data pipeline to the on-premises data centre, paired with an encrypted *AWS Site-to-Site VPN* configured as an active-passive fallback route.
- **Rationale:** Standard internet routing is too unpredictable for low-latency financial systems. *AWS Direct Connect* provides the reliable, sub-30ms throughput needed for synchronous transactional calls, while the automated BGP failover to VPN ensures the line stays up if the physical cable is cut.

5. Identity Propagation via Cognito to IBM RACF Mapping

- **Decision:** Authenticate end-users at the edge using *AWS Cognito* JWTs, and systematically map those cloud tokens to verified mainframe system IDs using *IBM RACF* (Resource Access Control Facility) at the *z/OS Connect* boundary.
- **Rationale:** This maintains end-to-end security governance. It prevents the architecture from using a single generic "admin" service account to execute all mainframe actions, ensuring that every cloud-initiated transaction is strictly tied to a specific user identity for compliance auditing.

6. Edge Threat Scraping via AWS WAF and Shield

- **Decision:** Route all public inbound client traffic through *AWS WAF* and *AWS Shield* before it touches the *Network Load Balancer (NLB)* or private application subnets.
- **Rationale:** Protecting the backend infrastructure requires stopping malicious activity early. Mitigating application exploits (*SQL injection, XSS*)

and high-volume *Distributed Denial of Service* (DDoS) attacks at the cloud edge ensures bad traffic is scrubbed completely before wasting costly internal cloud or mainframe compute power.

7. Containerized Compute Architecture via Amazon ECS

- **Decision:** Deploy core digital business logic and fintech services inside microservices hosted on *AWS ECS*, rather than utilizing a fully serverless *AWS Lambda* architecture for the primary application layer.
- **Rationale:** While *AWS Lambda* works perfectly for small, event-driven tasks, managing continuous, high-volume hybrid connection pools over Direct Connect to the mainframe requires long-lived server processes. *AWS ECS* containers minimize the risk of connection-timeout errors and cold-start latencies.

8. Network Micro-Segmentation within VPC Subnets

- **Decision:** Segregate the cloud architecture into isolated subnets, placing *AWS RDS* and the *OpenShift. z/OS Connect* integration layer into private-only subnets that lack any direct internet routing paths.
- **Rationale:** Minimizing the system's attack surface is a core zero-trust requirement. If a public-facing container in the ingress zone is compromised, lateral network movement is blocked, preventing attackers from reaching the integration pipelines that connect directly to the core mainframe database systems.

9. Distributed Tracing using OpenTelemetry across Cloud and Mainframe

- **Decision:** Enforce a unified distributed tracing standard using OpenTelemetry, injecting a unique correlation ID into every request at the cloud edge and mapping it to *IBM z/OS System Management Facilities* (SMF) logs.
- **Rationale:** Troubleshooting failures in a hybrid system is highly complex. A single unified correlation ID allows operations teams to easily trace a failed transaction from a user's mobile screen, through cloud *subnets*, down to the exact block of *COBOL* code executed on the mainframe.

10. Multi-AZ Active-Active Deployment with Regional Pilot Light

- **Decision:** Deploy all cloud applications actively across multiple isolated *AWS Availability Zones (AZs)*, backed by an automated Pilot Light disaster recovery stack in a separate geographical *AWS region*.
- **Rationale:** Financial services require continuous availability. Spreading containers across localized availability zones ensures instant failover if a single data centre loses power, while the secondary region guarantees business continuity during a massive, widespread cloud infrastructure failure.

AWS Well-Architected Framework Review

An evaluation of this architecture against the design principles of the AWS Well-Architected Framework reveals the following target alignments and gaps:

1. Security Pillar

- **Strengths:** The architecture implements robust edge defense by using *AWS Route 53*, *AWS CloudFront*, *AWS WAF*, and *AWS Shield* to scrub Layer 3/4/7 threats and DDoS attacks before they reach private subnets. Inside the network, a zero-trust model enforces micro-segmentation by placing *AWS RDS* and the IBM Z Modernization Stack (*IBM z/OS Connect*) in private *subnets*, while securing hybrid transit data with dual-layer encryption via *AWS Direct Connect* wrapped in an *IPsec VPN* tunnel.
- **Recommendations:** Perform regular deep-dive security audits on the token propagation and identity federation pipeline where *AWS Cognito* short-lived JWT tokens are mapped to native on-premises *IBM RACF* user profiles. This step is critical to prevent privilege drift, eliminate the risk of shared-account blind spots, and ensure complete compliance with financial data security regulations.

2. Reliability Pillar

- **Strengths:** High availability is baked into the compute and data layers by deploying *AWS ECS*, *Red Hat OpenShift*, and *AWS ElastiCache* across multiple isolated *Availability Zones (AZs)* backed by an automated 60-second *AWS RDS* database failover mechanism. On the hybrid layer, network redundancy is guaranteed by a dedicated *AWS Direct Connect* pipeline with an active-passive fallback *AWS Site-to-Site VPN*, while the

mainframe layer uses continuous hardware-level storage mirroring to match the primary data centre to a secondary DR site.

- **Recommendations:** Establish a schedule of quarterly chaos engineering drills to simulate a complete physical line cut and validate the BGP autonomous re-routing failover time. These tests are essential to ensure the automated failover pathways meet your strict target Metrics (RTO under 4 hours, near-zero RPO) without introducing data corruption into the core transactional ledger.

3. Cost Optimization Pillar

- **Strengths:** The design delivers significant mainframe cost savings by routing high-frequency, non-transactional read queries directly into an *AWS ElastiCache* tier, effectively shielding the mainframe from expensive processing unit (*MIPS*) scaling spikes. Additionally, using *AWS Lambda* for lightweight, event-driven tasks and applying automated scaling policies to the *AWS ECS* container workloads prevents the organization from paying for idle server time.
- **Recommendations:** Build a unified financial operations (FinOps) dashboard to calculate and track the holistic Total Cost of Ownership (*TCO*). This monitoring is required to verify that the ongoing cloud operational costs—including *Multi-AZ* fees, enterprise *OpenShift* licensing, and dedicated physical line rentals—do not eclipse the financial savings generated by reducing mainframe *MIPS* consumption.

4. Performance Efficiency Pillar

- **Strengths:** The solution minimizes processing delays by running long-lived *AWS ECS* container connection pools to avoid serverless cold starts, while deploying *IBM z/OS Connect* on *Red Hat OpenShift* to handle fast, automated JSON-to-binary data serialization. Global static assets are further accelerated at the cloud edge using *AWS CloudFront* to lower overall client round-trip latency.
- **Recommendations:** Establish strict Caching Time-to-Live (TTL) policies and data pre-fetching routines within the *AWS ElastiCache* layer. This optimization is necessary to maximize local cache hit ratios, counteract the physical network distance latency added by the hybrid cloud-to-ground jump, and keep end-to-end response times within enterprise SLAs.

5. Operational Excellence Pillar

- **Strengths:** Infrastructure management is modernized by using containerized pipelines for rapid software deployments, while *AWS Network Load Balancers* and *AWS Route 53* health check probes continuously monitor application pathways to automatically bypass faulty or degraded components.
- **Recommendations:** Implement a unified distributed tracing system using the OpenTelemetry standard to bridge the visibility gap caused by fragmented logging platforms. Injecting a unique correlation ID at the edge allows operations teams to trace transactions seamlessly across *AWS CloudWatch/AWS CloudTrail* and on-premises mainframe *System Management Facilities (SMF)* logs.

6. Sustainability Pillar

- **Strengths:** The pattern reduces environmental impact by running dense container workloads on *AWS ECS* to maximize cloud server compute utilization and efficiency. It further shrinks the physical hardware footprint, power usage, and cooling demands of the on-premises enterprise data center by offloading high-volume processing tasks and historical log data to the cloud.
- **Recommendations:** Create automated *AWS S3* Lifecycle policies that instantly move older log archives, audit trails, and non-relational files into colder storage tiers like *AWS S3 Glacier Deep Archive*. This automation lowers long-term cloud storage energy consumption and aligns the entire hybrid platform with corporate green-computing and carbon-reduction goals.

Architecture Risks

1. The Mainframe Connection Pool Exhaustion

- **The Flaw:** High-volume, short-lived *AWS ECS* container tasks open new TCP/IP sockets to *z/OS Connect* instead of using reusable connection channels.
- **The Risk:** The legacy mainframe exhaustion limits are breached during peak mobile traffic spikes, dropping new client transaction handshakes and causing widespread timeout errors.

- **The Fix:** Enforce aggressive connection pooling configurations within the *Red Hat OpenShift* tier and configure fixed-size socket reuse quotas for all containerized microservices.

2. The Cloud-Mainframe Log Fragmentation

- **The Flaw:** *AWS CloudWatch* holds the cloud application traces, while on-premises *System Management Facilities* (SMF) logs retain the transactional ledger details, with no shared indices.
- **The Risk:** Production troubleshooting teams are blinded during multi-system failures, leading to delayed root-cause discovery and a massive increase in Mean Time to Resolution (MTTR).
- **The Fix:** Implement a unified distributed tracing standard using OpenTelemetry, injecting a unique correlation ID at *AWS CloudFront* that is passed natively down to the mainframe database layer.

3. The Uncached Balance-Query Cost Spill

- **The Flaw:** Misconfigured cache invalidation rules or missing Time-to-Live (TTL) settings in *AWS ElastiCache* force repetitive user balance inquiries to pass through to the core database.
- **The Risk:** The enterprise encounters severe financial cost overruns due to unexpected and highly volatile mainframe *MIPS* scaling spikes triggered by simple, non-transactional read queries.
- **The Fix:** Hardcode strict 60-second caching TTL policies at the cloud edge and configure *AWS RDS* to serve as a read-only replica for non-real-time user profiling data.

4. The Shared Service Account Permission Drift

- **The Flaw:** *z/OS Connect* relies on a small handful of generic, highly privileged system service accounts to execute diverse transactional payloads inside *CICS TS* and *IMS TM*.
- **The Risk:** Malicious horizontal movement inside the cloud network allows an attacker to manipulate core financial records without leaving an auditable user trail, violating compliance regulations.
- **The Fix:** Map *AWS Cognito* user identities dynamically to distinct, low-privilege *IBM RACF* user IDs at the integration gateway boundary using token propagation protocols.

5. The Split-Brain Asynchronous Update Failure

- **The Flaw:** A dual-write operational approach updates application states in *AWS RDS* and the core *IBM Db2* mainframe engine at the same time without an explicit distributed transaction coordinator.
- **The Risk:** A transient networking blip on the *AWS Direct Connect* circuit causes the cloud database and the core ledger to fall out of sync, producing severe financial accounting errors.
- **The Fix:** Establish a strict "Mainframe-as-Master" data pattern where state-changing transactions commit to the mainframe ledger first, subsequently driving asynchronous cache updates down to the cloud.

6. The Single Direct Connect Fiber Cut Failure

- **The Flaw:** The hybrid network infrastructure relies on a single physical *AWS Direct Connect* circuit routing back to the primary on-premises *Customer Gateway*.
- **The Risk:** Accidental physical fiber damage at the local carrier exchange breaks the cloud-to-ground pipeline completely, cutting off the mobile application layer from the transaction core.
- **The Fix:** Configure an active-passive fallback path utilizing an encrypted *AWS Site-to-Site VPN* tunnel configured with automated *BGP routing* policies for sub-minute failover switching.

7. The Container Cold-Start Latency Accumulation

- **The Flaw:** Microservices that handle heavy core transaction translations utilize highly unoptimized runtime engines or are built inside massive, unpruned container image layers.
- **The Risk:** Sudden auto-scaling events inside *AWS ECS* trigger container launch delays, causing cascading network timeouts back to the *AWS Network Load Balancer* under heavy load.
- **The Fix:** Optimize all microservices using lightweight container baselines, pre-compile runtimes, and utilize long-lived task deployments to bypass cold-start compute tax entirely.

8. The Ingress API Payload Inflation Attack

- **The Flaw:** The private application subnets accept raw incoming JSON payloads from the ingress layer without setting hard character validation limits or structural boundary rules.

- **The Risk:** Attackers submit deeply nested, oversized JSON files that saturate the *IBM z/OS Connect* parsing engines, driving high memory usage and creating a Denial of Service (DoS) on the gateway.
- **The Fix:** Deploy strict payload size constraints and structural validation checks directly at the *AWS WAF* layer to instantly drop non-compliant API calls at the perimeter.

9. The Mainframe Disaster Recovery Data Gap

- **The Flaw:** The asynchronous storage replication link connecting the primary data centre to the secondary on-premises disaster recovery site lags under extreme transaction periods.
- **The Risk:** A sudden data center power failure causes uncommitted transactions to drop during DR activation, creating a costly and compliance-breaking Recovery Point Objective (RPO) data loss.
- **The Fix:** Upgrade to high-speed, synchronous hardware-level storage mirroring for core *IBM Db2* volumes and implement automated database transaction log scrubbers to verify ledger consistency.

10. The Static Database Credential Leak

- **The Flaw:** Mainframe connection strings and relational database credentials are saved inside text-based configuration profiles or embedded directly into *AWS ECS* task definitions.
- **The Risk:** Unauthorized access to internal code repositories exposes critical core database entry keys, allowing lateral administrative system exploitation across the cloud network.
- **The Fix:** Mandate the use of *AWS Secrets Manager* to store all backend database credentials, enforcing automatic 30-day credential rotation cycles without manual application downtime.

Architectural Recommendations

1 . Implement Distributed Tracing

- **Action:** Inject a unique W3C-compliant correlation ID at *AWS CloudFront* and propagate it across all application hops.
- **Recommendation:** Feed these IDs directly into a unified observability platform to bridge *AWS CloudWatch* and mainframe *System Management Facilities (SMF)* logs for end-to-end transaction visibility.

2. Harden the Identity Federation Layer

- **Action:** Connect *Amazon Cognito* token validation routines directly to the *IBM z/OS Connect* identity mapping configuration interface.
- **Recommendation:** Map short-lived cloud JWTs directly to specific, low-privilege on-premises *IBM RACF* user profiles to eliminate generic service accounts and maintain audit clarity.

3. Establish a Caching Strategy

- **Action:** Deploy strict 60-second Time-to-Live (TTL) rules inside *AWS ElastiCache* for high-frequency, non-transactional lookups.
- **Recommendation:** Route all digital balance and profile inquiries directly to this local cloud cache to intercept non-transactional traffic and lower expensive mainframe *MIPS* scaling costs.

4. Configure Network Path Redundancy

- **Action:** Deploy an encrypted *AWS Site-to-Site VPN* as an active-passive fallback alongside your *AWS Direct Connect* fiber circuit.
- **Recommendation:** Utilize *Border Gateway Protocol (BGP)* dynamic routing to ensure automated, sub-minute traffic switching if a physical network cable is accidentally severed.

5. Enforce Network Segmentation

- **Action:** Isolate *AWS RDS* and the *IBM Z Modernization Stack* containers inside private-only subnets within your *VPC*.
- **Recommendation:** Configure explicit Security Group rules that drop all incoming traffic to these subnets except for direct API calls from verified *AWS ECS Fintech Services*.

6. Externalize Secrets Management

- **Action:** Remove hardcoded connection strings and passwords from container code and task configurations.
- **Recommendation:** Mandate that all microservices fetch credentials dynamically from *AWS Secrets Manager*, and enable automated 30-day credential rotation cycles to secure backend systems.

7. Enforce API Payload Validation

- **Action:** Configure structural constraint rules and strict message size limits directly on *AWS WAF* at the network perimeter.
- **Recommendation:** Instantly drop oversized or malformed JSON payloads at the edge to protect *IBM z/OS Connect* parsing engines from malicious memory saturation attacks.

8. Optimize Connection Pooling

- **Action:** Configure persistent, long-lived TCP/IP connection pools within Red Hat OpenShift and *AWS ECS*.
- **Recommendation:** Reuse sockets for cloud-to-mainframe API handshakes to eliminate connection setup delays and prevent legacy mainframe gateway exhaustion under heavy user traffic.

9. Automate Infrastructure Deployments

- **Action:** Write and maintain all *AWS networking*, container, and security resources inside Terraform or *AWS CloudFormation* templates.
- **Recommendation:** Sync cloud Infrastructure as Code (IaC) deployment pipelines with mainframe change management schedules using automated orchestration tools to ensure cross-platform environment consistency.

10. Implement a "Mainframe-as-Master" Write Pattern

- **Action:** Block cloud applications from executing direct, local transactional updates for core accounting states inside *AWS RDS*.
- **Recommendation:** Enforce an architecture rule where all state-changing writes must commit to the *IBM z/OS Mainframe* databases first, subsequently driving asynchronous cache updates down to the cloud layer.

Boundary Conditions for Reference Architecture

1: Maximum Hybrid Network Latency Threshold

- **The Issue:** Transient network congestion or bad routing over the *AWS Direct Connect* link causes API response times to spike, threatening to freeze customer-facing mobile applications.
- **The Condition:** If the round-trip network latency between *IBM z/OS Connect* on AWS and the on-premises *Customer Gateway (CGW)* exceeds

150 milliseconds, the cloud container must instantly trip its circuit breaker, fail fast locally, and serve gracefully degraded data from *AWS ElastiCache*.

2. Max Cache-Miss Volume on Mainframe Compute (MIPS Cap)

- **The Issue:** A sudden rush of users or a faulty code deployment creates a massive wave of cache misses in *AWS ElastiCache*, flooding the mainframe with direct read queries and causing catastrophic processing cost spikes.
- **The Condition:** When direct API read requests to the *IBM z/OS Mainframe* exceed 2,500 requests per second (RPS), the cloud *AWS Network Load Balancer* (NLB) must aggressively throttle incoming read traffic at the edge, preserving core mainframe compute power for critical transactional writes.

3. Payload Size Constraints for Protocol Transformation

- **The Issue:** A client or partner application attempts to upload an oversized, deeply nested JSON file, threatening to exhaust the memory buffers of the *IBM z/OS Connect* parsing engines during binary translation.
- **The Condition:** The *AWS WAF* layer must enforce a hard packet restriction that instantly drops any inbound API payload exceeding 10 Megabytes (MB), completely preventing unvalidated or oversized arrays from reaching the integration layer.

4. Connection Pool Saturation Limit

- **The Issue:** A massive spike in synchronous transactional write requests exhausts all open TCP/IP sockets between *Red Hat OpenShift* and *CICS TS / IMS TM*, which can crash the hybrid gateway.
- **The Condition:** If the active connection pool utilization hits 95% of the maximum allocated sockets, the integration layer must immediately reject new non-essential traffic, queue low-priority requests, and raise an critical alert to spin up additional pre-warmed container routes.

5. Disaster Recovery Data Desynchronization Lag

- **The Issue:** Under extreme transaction volume, the asynchronous storage replication link between the primary on-premises data centre and the secondary DR site falls behind, widening the data gap.
- **The Condition:** If the data synchronization lag between the primary *IBM Db2 / VSAM* volumes and the standby DR site exceeds a 10-second

threshold, the system must trigger an automated operations alert and pause non-critical cloud batch updates until the core ledger replication safely catches up.

Appendix A: Non-Functional Requirements (NFR) Matrix

This matrix defines the critical quality attributes, operational constraints, and performance benchmarks that the system must satisfy to ensure long-term stability, security, and scalability.

NFR Category	Target Objective	Engineering Validation Mechanism(Tools)	Architecture Component Mapping
Availability	• Cloud Tier: 99.99% uptime. • Mainframe Tier: 99.999% uptime	• AWS Fault Injection Service (FIS), • IBM Parallel Sysplex cluster validation.	Amazon ECS, Red Hat OpenShift, IBM z/OS Mainframe.
Latency (Query & API)	• Local Cache Read: <15ms. • Hybrid Mainframe Write: <150ms.	• Amazon CloudWatch Synthetics, AWS X-Ray, AppDynamics.	Amazon ElastiCache, z/OS Connect, AWS Direct Connect.
Data Ingestion Sync Latency	• Asynchronous cache invalidation: <2 seconds. • Mainframe DR replication lag: <10 seconds.	• Custom Prometheus metrics exporter monitoring transactional timestamps.	IBM Db2, VSAM, Amazon S3, Amazon RDS.
Data Protection	• Zero cleartext transit data. • AES-256 at-rest protection.	• AWS Security Hub • IBM Crypto Express diagnostic panels.	AWS KMS, AWS Direct Connect, IPsec VPN, IBM Db2.
Perimeter Security	• 100% automated edge threat scrubbing. • Payload size restriction <10MB.	• AWS WAF logs • AWS Shield monitoring dashboards.	Amazon CloudFront, AWS WAF, AWS Shield, Network Load Balancer (NLB).
Identity & Access	• Short-lived token validation (<15 mins). • Zero persistent hardcoded credentials	• AWS IAM Access Analyzer • IBM RACF security audit logs.	Amazon Cognito, AWS Secrets Manager, IBM RACF.
Throughput Capacity	• Edge Handling: Up to 15,000 RPS. • Mainframe Limit: Cap at 2,500 RPS read traffic.	• Apache Jmeter AWS Distributed Load Testing Solution.	Amazon ECS, Network Load Balancer, z/OS Connect.

Data Retention	• Cloud Cache/State Logs: 30 days. • Core Financial Ledgers: 7+ years (regulatory compliant).	• Amazon S3 Lifecycle configurations, • IBM DFSMS retention rules.	Amazon S3 Glacier, IBM Db2, VSAM.
System Availability	• Uninterrupted service during single AZ failures.	• AWS Auto Scaling tests, simulated zone-isolation blackouts.	Multi-AZ Amazon ECS, Multi-AZ Amazon RDS.
Disaster Recovery	• RTO < 4 hours. • RPO ~ 0 (near-zero data loss).	• Automated DR drills, AWS Backup validation scripts.	AWS Site-to-Site VPN, Backup BGP Routers, Secondary DR Mainframe.
Compute Efficiency	• Cloud Container Utilization: 70–80% CPU/Mem range. • Mainframe MIPS optimization.	• Amazon ECS Auto Scaling metrics • IBM Resource Measurement Facility (RMF).	Amazon ECS tasks, IBM zSystems Mainframe processors
Storage Optimization	• Local database access <10ms. • Automatic tiering of old assets.	• Amazon RDS • Performance Insights, • S3 Storage Lens.	Amazon RDS, Amazon S3 Lifecycle, IBM DFSMS.
Regulatory Audit	• Immutable, non-repudiable transaction trails for compliance (PCI-DSS / SOC 2).	• AWS CloudTrail insights • IBM System Management Facilities (SMF) analysers.	AWS CloudTrail, IBM SMF logging blocks.
Observability Logs	• Unified correlation across hybrid components within 5 seconds of log ingestion.	• OpenTelemetry (OTel), • Dynatrace • Splunk log aggregator.	AWS CloudWatch, z/OS Connect, Mainframe SMF.

Appendix B: Cost Optimization & Attribution Matrix

This appendix provides a structured evaluation framework for linking operational expenses to their true business drivers, ensuring financial accountability and resource efficiency.

Cost Domain	Identified Bleed Point	Architectural Fix / Optimization Strategy	Expected Financial Impact
Mainframe Compute	 OpEx	Intercept traffic at the cloud edge using Amazon ElastiCache with a strict 60-second TTL policy to fulfill non-transactional queries.	Reduces on-premises mainframe MIPS consumption costs by up to 40% during peak user hours.
Hybrid Connectivity	 CapEx	- Deploy an AWS Site-to-Site VPN during initial testing phases - Scale up to physical AWS Direct Connect lines only when production traffic benchmarks demand it.	Saves immediate upfront networking procurement costs and lowers early-stage project expenditure.
Data Storage	 OpEx	Implement automated Amazon S3 Lifecycle policies to instantly offload and archive older logs directly into S3 Glacier Deep Archive.	Lowers on-premises storage array leasing costs and reduces active database maintenance overhead.
Cloud Compute	 OpEx	Configure target-tracking auto-scaling policies on Amazon ECS to dynamically adjust the container footprint based on real-time CPU utilization.	Eliminates waste from idle cloud servers, shrinking the monthly cloud compute bill by 20–30%.
Platform Licensing	 CapEx	Utilize a tiered deployment roadmap, starting with a minimal container footprint for z/OS Connect and scaling node counts dynamically as APIs go live.	Defers heavy upfront licensing costs and aligns platform software expenses directly with project production

Database Compute	 OpEx	• Enable Amazon RDS Read Replicas in separate availability zones to offload reporting, keeping the primary database light and properly sized.	• Prevents the need to scale up to larger, more expensive cloud database instance tiers.
Network Egress	 OpEx	• Enforce strict payload compression (Gzip) and streamline API data structures inside IBM z/OS Connect to minimize message sizes.	• Reduces ongoing hybrid data transfer charges over the AWS Direct Connect pipeline.
Non-Production Environments	 OpEx	• Implement automated AWS Instance Scheduler scripts to shut down non-production Amazon ECS tasks and Red Hat OpenShift worker nodes outside core business hours (e.g., 7 PM to 7 AM and weekends).	• Reduces non-production compute infrastructure spend by up to 60-70% monthly.
Secrets & Encryption Management	 OpEx	• Configure local, secure in-memory caching of rotated keys and database credentials inside the Amazon ECS Fintech Services container task memory layer.	• Eliminates excessive per-call API request fees from AWS Secrets Manager and AWS KMS during high-throughput peaks.
Global Traffic Routing	 CapEx	• Delegate all geo-proximity routing, global server load balancing (GSLB), and early health checking to Amazon Route 53 and Amazon CloudFront at the cloud edge.	• Removes the upfront procurement, configuration, and data center rack space costs associated with traditional networking hardware appliances.

Appendix C: Enterprise Security Risk Register

This register serves as the central repository for identifying, assessing, and tracking security vulnerabilities and threats across the organization, enabling leadership to make informed, risk-based decisions.

Security Vulnerability Description	Likelihood (1-5)	Impact (1-5)	Risk Rating (1-25)	Core Technical Mitigation Strategy
Mainframe Service Account Credential Theft: Compromise of static, high-privilege mainframe database strings or connection keys stored inside cloud code repositories or application files.	2	5	10 (Medium)	Mandate the use of AWS Secrets Manager to store all backend database credentials, enforcing automatic, programmatic 30-day credential rotation cycles.
API Payload Inflation & Memory Saturation: Attackers submit oversized or deeply nested JSON payloads designed to exhaust the memory buffers of z/OS Connect parsing engines.	3	4	12 (Medium)	Enforce strict request size limits and structural schema validations directly at the AWS WAF layer to instantly drop non-compliant calls at the network boundary.
Lateral Movement via Network Ingress Breach: A compromise of a public-facing cloud container allows an attacker to move laterally into the internal pipelines connecting to the mainframe database.	2	5	10 (Medium)	Enforce network micro-segmentation by isolating the IBM Z Modernization Stack in private subnets, dropping all traffic except explicit calls from verified Amazon ECS Tasks.
Cleartext Financial Data Over-the-Air Leak: Interception of highly sensitive core banking and customer transaction payloads as they travel	1	5	5 (Low)	Enforce dual-layer transit encryption, routing all hybrid traffic over an AWS Direct Connect dedicated physical line wrapped inside an encrypted IPsec VPN tunnel.

across the hybrid cloud-to-ground boundary.				
Mainframe MIPS Denial of Service (DoS): An attacker or a rogue cloud application loops uncached read requests (e.g., rapid balance checks), intentionally crashing or driving massive cost spikes on the mainframe.	4	3	12 (Medium)	Deploy strict 60-second caching TTL rules inside Amazon ElastiCache and implement automated, aggressive rate-limiting throttles on the Network Load Balancer (NLB).
Broken Object-Level Authorization (BOLA): A user manipulates authenticated account IDs within an API request parameter to view or modify another customer's core financial data.	3	4	12 (Medium)	Map Amazon Cognito user identities dynamically to verified IBM RACF user profiles at the integration boundary, validating that the token identity matches the requested account data.
Cryptographic Key Management Compromise: A breach of the cloud-tier master encryption keys exposes localized application states, data stores, and historical logging files.	1	5	5 (Low)	Isolate AWS KMS customer-managed keys completely from the master keys governing on-premises IBM Db2 storage layers, enforcing tight separation of cryptographic domains.
Orphaned Identity Token Exploitation: Attackers capture compromised long-lived API session tokens to execute administrative actions long after a legitimate user session has ended.	3	4	12 (Medium)	Enforce systematic directory-based partitioning on S3 and enforce workgroup spending ceilings and querying boundaries inside AWS Athena.
Tampering of Hybrid Transaction Audit Trails: An insider or compromise vector alters cloud or mainframe logging files to mask unauthorized system	2	4	8 (Low)	Stream all AWS CloudTrail and IBM System Management Facilities (SMF) logs continuously to an immutable, write-once-read-many

configuration changes or fraudulent writes.				(WORM) centralized log storage engine.
Gateway Socket Connection Exhaustion: An attacker floods the network with slow, incomplete API handshakes to occupy all available hybrid TCP/IP sockets and freeze legitimate transaction processing.	3	3	9 (Medium)	Configure strict connection timeouts and fixed reuse quotas within the Red Hat OpenShift tier, dropping unestablished or sluggish socket allocations aggressively.

Appendix D: Enterprise RACI Matrix

This matrix defines the clear operational boundaries, decision rights, and delivery expectations across cross-functional teams to eliminate role confusion and accelerate project execution.

Key Architectural Deliverable	Cloud Infra Team	Mainframe Ops & Infra Team	Enterprise Cyber Security	FinOps	Dev	DevOps & SRE Team
Designing Hybrid Network Paths (Direct Connect/VPN)	<i>R</i>	<i>R</i>	<i>C</i>	<i>I</i>	<i>I</i>	<i>C</i>
Provisioning Integration Layer (z/OS Connect/OpenShift)	<i>R</i>	<i>C</i>	<i>C</i>	<i>I</i>	<i>I</i>	<i>R</i>
Mapping Identities (Cognito to IBM RACF Mapping)	<i>C</i>	<i>R</i>	<i>A</i>	<i>I</i>	<i>C</i>	<i>I</i>
Configuring Cache Policies (ElastiCache TTL / MIPS Limits)	<i>R</i>	<i>C</i>	<i>I</i>	<i>C</i>	<i>R</i>	<i>C</i>
Writing Core APIs & Serialization (COBOL to JSON)	<i>I</i>	<i>R</i>	<i>I</i>	<i>I</i>	<i>R</i>	<i>I</i>
Enforcing Edge & Perimeter Defenses (WAF/Shield/NLB)	<i>R</i>	<i>I</i>	<i>A</i>	<i>I</i>	<i>I</i>	<i>R</i>
Managing Secrets & Rotation Keys (Secrets Manager)	<i>R</i>	<i>C</i>	<i>A</i>	<i>I</i>	<i>C</i>	<i>R</i>
Executing Hybrid DR Drills (RTO/RPO Validation)	<i>R</i>	<i>R</i>	<i>C</i>	<i>I</i>	<i>I</i>	<i>A</i>
Tuning Core Compute Efficiency (MIPS Capping/Auto-scaling)	<i>C</i>	<i>R</i>	<i>I</i>	<i>A</i>	<i>I</i>	<i>R</i>
Setting Up Unified Logs (OpenTelemetry Telemetry Stream)	<i>R</i>	<i>C</i>	<i>C</i>	<i>I</i>	<i>C</i>	<i>A</i>

About the Author

Amit Kulkarni

Senior Solutions Architect

📍 Pune, India | ✉ amitkwork2920@gmail.com | 🔗 [LinkedIn](#) | 💻 [Github](#)

I'm a Technology-agnostic architecture leader having 18+ years of partnership with 15+ clients across BFSI, Retail, Insurance, Healthcare, Travel, Media, and the Public Sector. Proven track record in cloud transformation (AWS, Azure), legacy COTS and mainframe modernization, security posture assessments, microservices modernization, event-driven architectures, and working knowledge on AI initiatives.

Architectural Specialization (Recent Focus)

During a recent deliberate transition period away from formal corporate employment, I dedicated my time to advanced deep-dives into enterprise cloud topologies, edge cases, and failure domains. This case study on *Legacy MainFrame Hybrid Modernization on AWS* is part of a broader body of architectural research aimed at critiquing standard vendor blueprints, identifying hidden system vulnerabilities, and building production-ready engineering remediations for high-throughput transactional environments.

I am currently seeking my next high-impact engineering role where I can solve complex distributed systems challenges and help scale robust, mission-critical infrastructure teams.